



第3回 Pythonによる科学計算(Numpy)

熊澤倫之

以下の行為はご遠慮ください。
ご協力よろしくお願いいたします。

- ・ 講義動画の録画・録音
- ・ 講義資料の無断の二次利用・配布
- ・ 講義内容やSlack上でのやりとりをSNS上でアップする行為

熊澤 倫之(くまざわ ともゆき)



所属

国内メーカー勤務

講師歴

GCI 2024winter Numpy

GCI 2025summer Numpy・教師あり学習

GCI 2025winter Numpy・教師あり学習

講義前に一言

初学者の方にも分かりやすいよう、特に重要なポイントと、今は概要の理解で十分な部分を明確にし、強弱をつけた講義を心がけます。

本日の講義の内容を1ページで解説

(本日の講義でできるようになること)

Numpyと呼ばれるライブラリ(ツールのようなもの)の基本機能が使えるようになる

1. Numpyとは何か、その特徴

2. 1次元配列

2-1. 計算の基本(ユニバーサル関数・ブロードキャスト・集約関数)

2-2. 中身のデータ参照方法(インデクシング)

3. 2次元配列

3-1. 計算の基本(ユニバーサル関数・ブロードキャスト)

3-2. 縦軸・横軸の概念(axis・集約関数)

3-3. 中身のデータ参照方法(インデクシング)

1.Numpyとは何か、その特徴

Numpy: Pythonで**複雑な科学計算**を行うためのライブラリ

Numpyの特徴

1次元配列

```
[0, 1, 2, 3, 4]
```

1. N次元配列を効率的に扱うことができる
2. for文なしで複雑な計算ができる(次ページ)
3. 計算が高速(Numpy内部は高速なC言語が主に使われている)
4. 科学計算のための豊富な関数がある

2次元配列

```
[[70, 95, 80],  
 [34, 42, 50],  
 [74, 37, 65]]
```

ライブラリ: プログラムを書く際に便利なツールや機能を集めたもの

Numpy: Pythonで**複雑な科学計算**を行うためのライブラリ

Numpyの配列 (ndarray) は、前回学習したPythonのリストと似ていますが、いくつか重要な違いがあります。

違いの例) ユニバーサル関数によって、
for文なしで計算ができます。

Numpyを使って計算する場合

input

```
result = a + b
```

a : [1, 2]

b : [3, 4]

→ [4, 6]

} 少ない記述で済む！

リストのまま計算する場合

input

```
result = []  
for i in range(len(a)): sum.append(a[i] + b[i])
```

}

Numpyの使い方① インポート方法

慣習により、Numpyは`np`という別名を付けて利用します。

以下のようにimportしてください。

input

```
import numpy as np
```

※ as ○○ で略称を指定する

Numpyの使い方② 関数(≡機能のようなもの)

np.○○(処理対象のオブジェクト)

様々な関数名を○○部分に指定して、()内のオブジェクトに処理を適用します。

例えば、以下のように使うことができます。

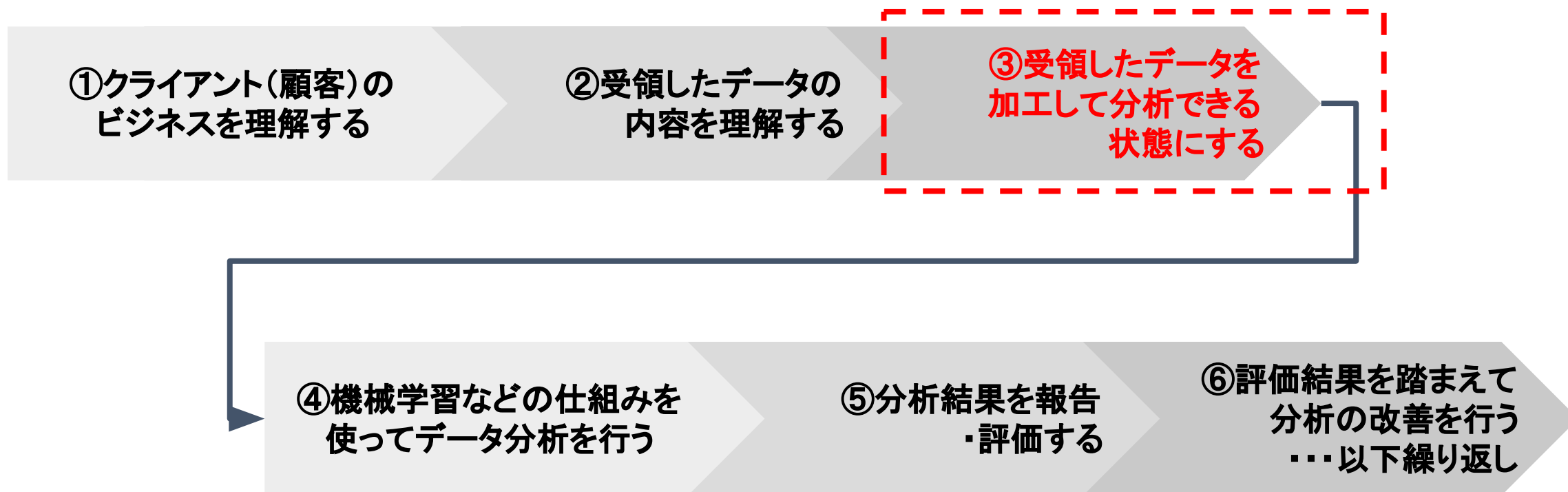
input

```
array_a = np.array([5, 6, 8])
```

データ分析にはデータ加工の工程があり、そのためにNumpyやPandasが必要

データ分析プロジェクトの一般的な流れ

高速な数値計算が得意なNumpyや、
データ加工・操作に特化したPandasが活躍します



Numpyはデータサイエンス・AI開発を行ううえで不可欠となるライブラリ



Numpy(本日のテーマ)

- 色々な計算をするための機能詰め合わせセット
- 今後登場するライブラリや、データ分析・AI開発でもほぼ間違いなく登場するため、学習必須

Pandas

- データ加工を得意とする機能詰め合わせセット

Matplotlib

- データを人間がわかりやすく図示するための機能詰め合わせセット

第2回から第5回まででPythonの各種便利機能の使い方を学び、それを踏まえてデータ分析の本丸である後半の教師あり学習に続きます

★都度調べればよい

まずNumpyの機能についてどこまで覚える必要があるのか？ということですが、これに関しては、覚えるべきことは少ないと思っています。

特に今回の講義だと、1次元、2次元配列の基本的な扱い方を覚えるだけでよく、他の機能については**その都度調べればよい**のです。

そうすることで、自然とよく使うものは覚える一方、滅多に使わないようなものをたくさん覚える必要もなくなります。実際プロのエンジニアでも自分が普段扱わない機能については知らないことはたくさんあり、その都度調べています。

調べながら学ぶ

関数(≡機能)は暗記するものではなく、**調べるものです**。
Numpyには(Numpyに限らず)膨大な数の関数が用意されています。

やってみたいことがあれば、公式ドキュメントなどで検索してみましょう！

本講義で扱うこと/扱わないこと

	扱う	扱わない
配列の次元	1次元・2次元配列	3次元以上の配列
内容の複雑度	Numpyの基礎的な操作（Numpy 公式ドキュメント参照）	Numpyを用いた複雑な操作（機 械学習モデルの定義など）
演習用notebook	GCIで今後も出てくる 基本的な操作や関数	練習問題 線形代数に関する詳しい解説

いま説明したのが赤枠部分

(本日の講義でできるようになること)

Numpyと呼ばれるライブラリ(ツールのようなもの)の基本機能が使えるようになる

1. Numpyとは何か、その特徴

2. 1次元配列

2-1. 計算の基本(ユニバーサル関数・ブロードキャスト・集約関数)

2-2. 中身のデータ参照方法(インデクシング)

3. 2次元配列

3-1. 計算の基本(ユニバーサル関数・ブロードキャスト)

3-2. 縦軸・横軸の概念(axis・集約関数)

3-3. 中身のデータ参照方法(インデクシング)

2.一次元配列

【復習】Pythonには、**リスト**というデータ型がある

リストとは、`[]`で囲われ、カンマで区切られた複数の値を持つ形式です。

リストは以下のように作ることができます。

input

```
list_a = [0, 1, 2, 3, 4]
```

```
list_a
```

output

```
[0, 1, 2, 3, 4]
```

Numpyの配列「ndarray」の作成

Numpyでは、**numpy.ndarray** というデータ構造で配列を扱います。

ndarrayは **np.array(リスト)** で作ることができます。

input

```
array_a = np.array([0, 1, 2, 3, 4])  
array_a
```

output

```
array([0, 1, 2, 3, 4])
```

ndarrayには、様々なNumpyの機能を適用できます。

リストとnumpy.ndarrayの比較

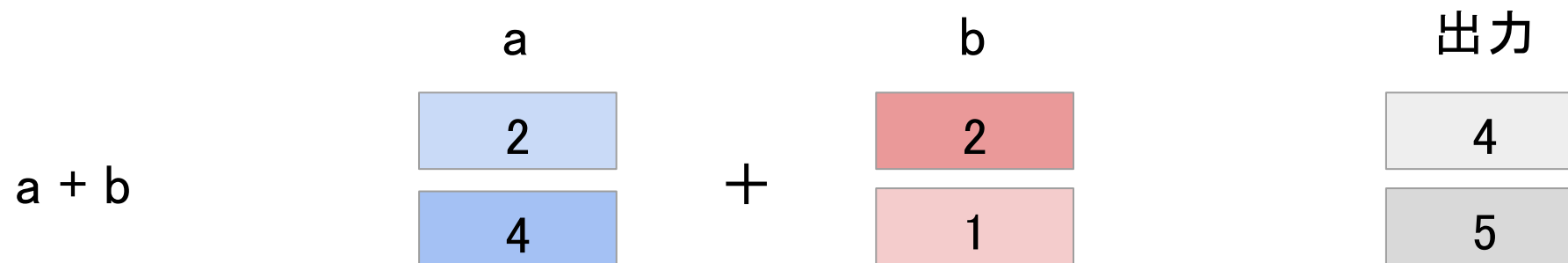
	リスト	numpy.ndarray
格納できる要素の型	異なる型の値を格納できる	同じ型の値しか格納できない
四則演算	リスト同士の四則演算は煩雑	ndarray同士の四則演算が容易
多次元配列の扱いやすさ	入れ子構造で多次元配列を表現できるが、操作が煩雑	axisという概念で効率的に多次元配列を扱える

ユニバーサル関数(ufunc): ndarrayを要素ごとに操作する関数

ndarrayは、**同じ位置の要素同士で演算を行う** ことができます。

例) 足し算

a, bはndarray



ndarrayの四則演算

引き算

`np.subtract(a, b)`

$a - b$

a
2
4

—

b
2
1

出力

0
3

掛け算

`np.multiply(a, b)`

$a * b$

a
2
4

*

b
2
1

4
4

割り算

`np.divide(a, b)`

a / b

a
2
4

/

b
2
1

1
4

ndarrayの四則演算

実際にコードを書くときは、以下のようにします。

例) 足し算 具体的な実装

input

```
a = np.array([2, 4])  
b = np.array([2, 1])  
a + b
```

output

```
array([4, 5])
```

【復習】リストの場合の「+」演算子

※注意 リストで要素同士の四則演算を行う場合はforループが必要 です。

2つのリストを「+」演算子で結合すると、リスト同士が連結されます。

input

```
a = [2, 4]
```

```
b = [2, 1]
```

```
a + b
```

output

```
[2, 4, 2, 1]
```

また、「-、*、/」などの算術演算子を使用するとエラーが発生します。

ndarrayの条件演算

等号

`np.equal(a, a)`

`a == a`

2
4

`==`

2
4

True

True

等号

`np.equal(a, b)`

`a == b`

a

2
4

`==`

b

2
1

True

False

大なり

`np.greater(a, b)`

※小なり `np.less(a, b)`

`a > b`

a

2
4

`>`

b

2
1

False

True

【復習】リストの場合の条件演算

※注意 リストで条件演算を行うと、リスト全体に対する結果を返します。

input

```
a = [2, 4]
```

```
b = [2, 1]
```

```
a == a
```

output

```
True
```

input

```
a == b
```

output

```
False
```

ユニバーサル関数: ndarrayの要素ごとに演算を行う関数

`a = np.array([1, 2])` とすると、以下のような出力が得られます。

指数関数

input

`np.exp(a)`

output

`array([2.71828183, 7.3890561])`

対数関数

input

`np.log(a)`

output

`array([0.0, 0.69314718])`

三角関数

※ cosやtanも同様

input

`np.sin(a)`

output

`array([0.84147098, 0.90929743])`

ブロードキャスト: 自動的に形状を合わせて演算を可能にする機能

スカラー(例: 個別の数字)が配列の形状に合わせて拡張されて、演算が行われます。

	a	スカラー		a	拡張後配列	出力
足し算	2 4	+ 3		2 4	3 3	5 7
掛け算	2 4	* 1.6		2 4	1.6 1.6	3.2 6.4

集約関数: Numpyの配列内の要素を1つの値に集約する関数

`a = np.array([1, 2, 3])` とすると、以下のような出力が得られます。

最大値

input

`np.max(a)`

output

3

合計値

input

`np.sum(a)`

output

6

平均値

input

`np.mean(a)`

output

2.0

標準偏差

input

`np.std(a)`

output

0.816496580927726

`np.arange(start, stop, step)`: start からstop未満のndarrayをstepの間隔で生成

input

`np.arange(8)`

output

`array([0, 1, 2, 3, 4, 5, 6, 7])`

input

`np.arange(2, 8)`

output

`array([2, 3, 4, 5, 6, 7])`

input

`np.arange(2, 8, 2)`

output

`array([2, 4, 6])`

少しNotebookをみておきましょう

いま説明したのが赤枠部分

(本日の講義でできるようになること)

Numpyと呼ばれるライブラリ(ツールのようなもの)の基本機能が使えるようになる

1. Numpyとは何か、その特徴

2. 1次元配列

2-1. 計算の基本(ユニバーサル関数・ブロードキャスト・集約関数)

2-2. 中身のデータ参照方法(インデクシング)

3. 2次元配列

3-1. 計算の基本(ユニバーサル関数・ブロードキャスト)

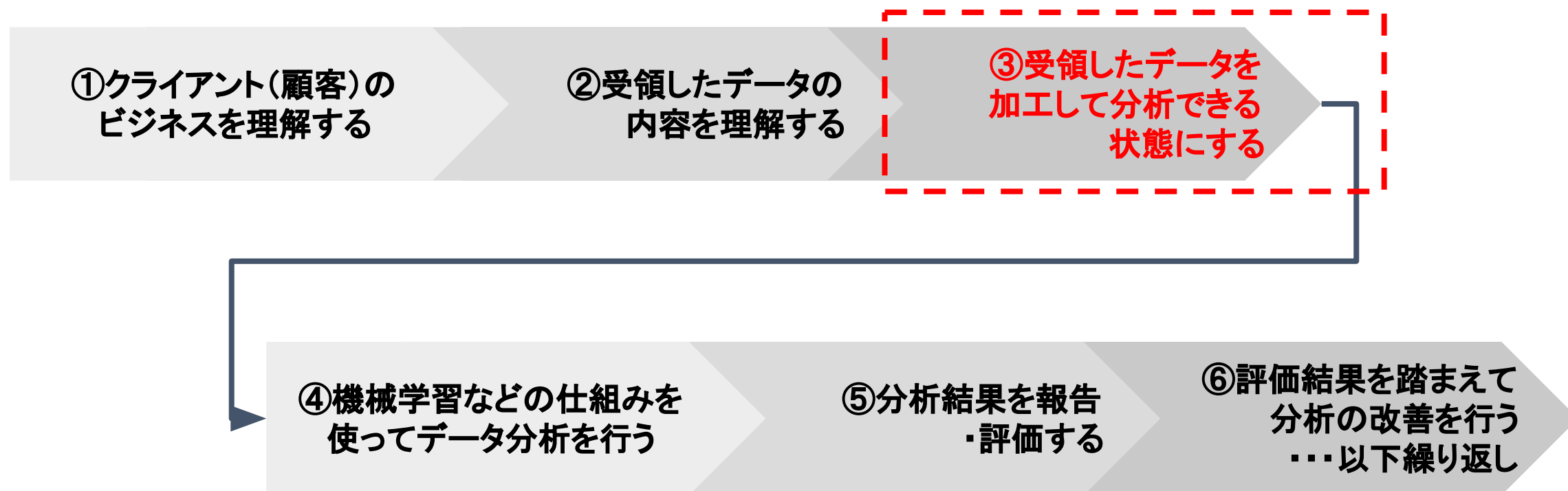
3-2. 縦軸・横軸の概念(axis・集約関数)

3-3. 中身のデータ参照方法(インデクシング)

データ分析にはデータ加工の工程があり、そのためにNumpyやPandasが必要

データ分析プロジェクトの一般的な流れ

高速な数値計算が得意なNumpyや、
データ加工・操作に特化したPandasが活躍します



冒頭クイズインデクシング(1)

インデクシングによって、30分ごとの風速データから値を取り出してみよう。

配列a 30分ごとの風速

12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00	20:30	21:00
[12,	13,	15,	14,	9,	11,	10,	13,	9,	7,	8,	-3,	-5,	-11,	-9,	-12,	-8,	0,	1]

Q1. 13:00のデータのみを取り出すには？

Q2. 13:00~16:00(未満)のデータのみを取り出すには？

Q3. 00分ちょうどのデータのみを取り出すには？



**インデクシングによって
取り出す**

インデクシング: ndarrayにインデックスを指定し、任意の要素を抽出

左から順に0, 1, 2, ...のインデックスを指定して任意の要素を抽出することができます。

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
配列a	0	3	6	9	12	15	18	21	24	27

input

```
a = np.array([0, 3, 6, 9, 12, 15, 18, 21, 24, 27])
```

```
a
```

output

```
array([0, 3, 6, 9, 12, 15, 18, 21, 24, 27])
```

インデクシング: ndarrayにインデックスを指定し、任意の要素を抽出

左から順に0, 1, 2, ...のインデックスを指定して任意の要素を抽出することができます。

1つの値のみを取り出したいときは、取り出したい要素のインデックスを指定します。

a[1]

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
0	3	6	9	12	15	18	21	24	27

input

a[1]

output

3

インデクシング: ndarrayにインデックスを指定し、任意の要素を抽出

右から順に-1, -2, -3, ...と指定して任意の要素を抽出することもできます。

一番最後の要素を取り出したい場合、-1を指定します。

a[-1]

[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
[0,	3,	6,	9,	12,	15,	18,	21,	24,	27]

input

a[-1]

output

27

インデクシング: ndarrayにインデックスを指定し、任意の要素を抽出

複数のインデックスを指定して値を取り出すこともできます。

欲しい要素のインデックスをリストで指定します。

a[[1, 4, 8]] ^[0] ^[1] ^[2] ^[3] ^[4] ^[5] ^[6] ^[7] ^[8] ^[9]
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27]

input

a[[1, 4, 8]]

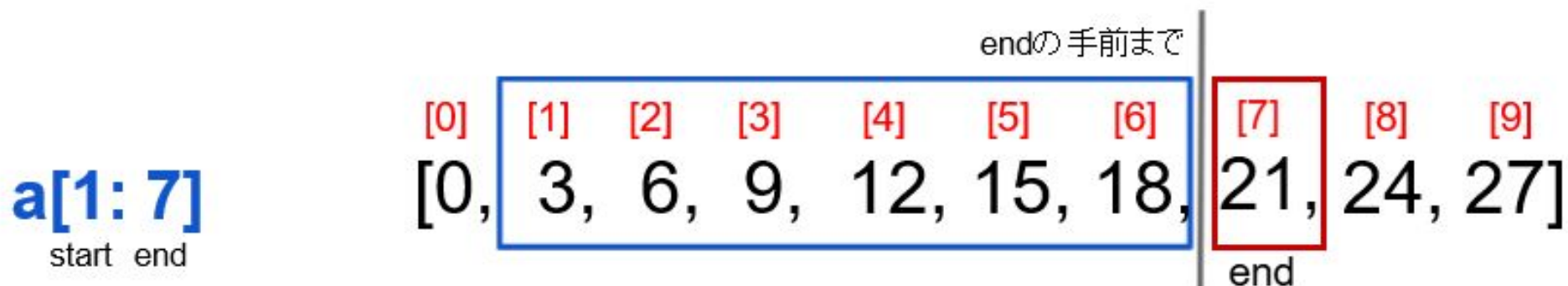
output

array([3,12,24])

ndarrayに[start: end: step]を指定し、任意の要素を抽出

連続したインデックスの値を取り出したいときは、startとendを指定します。

startとendの間には、スライス「:」を挟みます。



input

`a[1:7]`

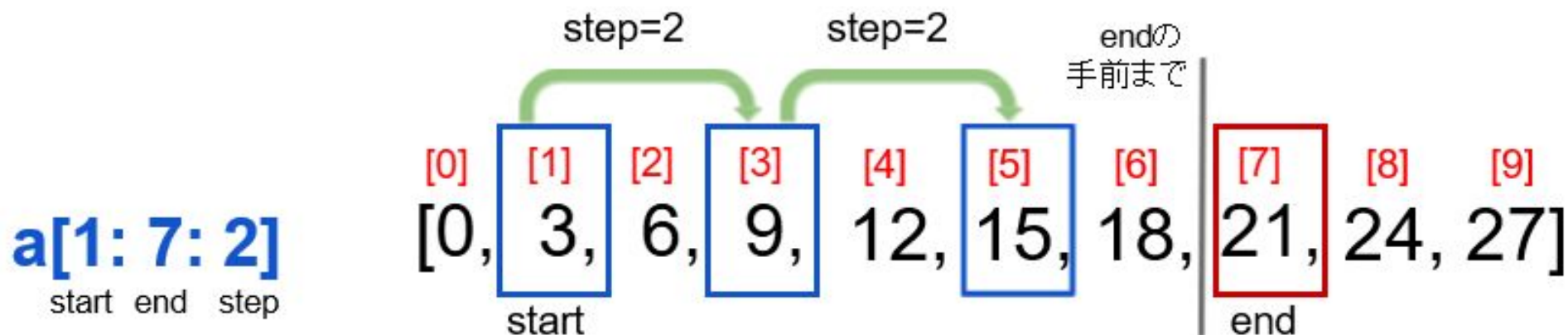
output

`array([3, 6, 9, 12, 15, 18])`

ndarrayに[start: end: step]を指定し、任意の要素を抽出

一定間隔で要素を取り出したいときは、start、end、stepを指定します。

startとendとstepの間には、スライス「:」を挟みます。



input

`a[1: 7: 2]`

output

`array([3, 9, 15])`

ndarrayに[start: end: step]を指定し、任意の要素を抽出

start以降の全ての値を指定したい場合はスライス「:」の後ろのendを省略できます。

a[1:]
start (end)

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
0	3	6	9	12	15	18	21	24	27

start

input

a[1:]

output

array([3, 6, 9, 12, 15, 18, 21, 24, 27])

ndarrayに[start: end: step]を指定し、任意の要素を抽出

end-1の全ての値を指定したい場合はスライス「:」の前のstartを省略できます。

a[: 7]
(start) end



input

a[: 7]

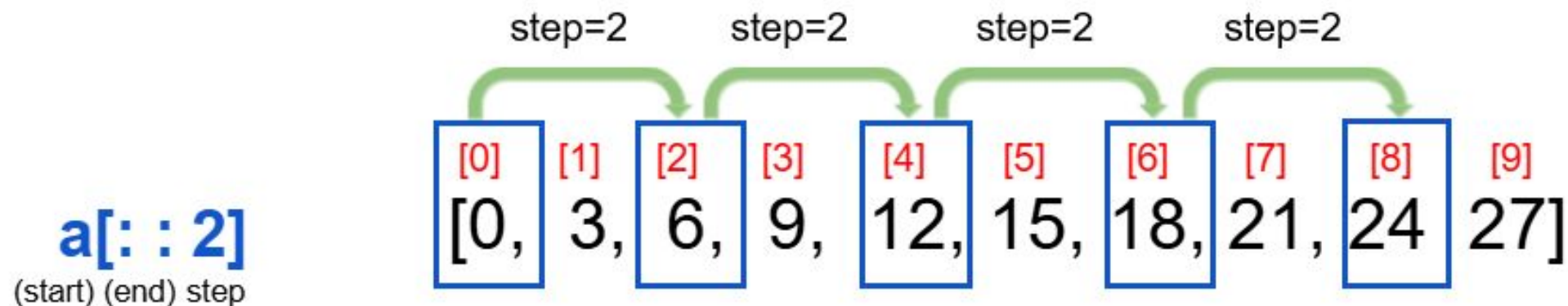
output

array([0, 3, 6, 9, 12, 15, 18])

ndarrayに[start: end: step]を指定し、任意の要素を抽出

配列から一定間隔で要素を取り出したいときは、stepを指定します。

startとendを省略すると、配列全体がスライスの対象になります。



input

a[::2]

output

array([0, 6, 12, 18, 24])

冒頭のクイズの答え

配列a 30分ごとの風速

12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00	20:30	21:00
[12,	13,	15,	14,	9,	11,	10,	13,	9,	7,	8,	-3,	-5,	-11,	-9,	-12,	-8,	0,	1]
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]

Q1. 13:00のデータのみを取り出すには？

冒頭のクイズの答え

配列a 30分ごとの風速

12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00	20:30	21:00
[12,	13,	15,	14,	9,	11,	10,	13,	9,	7,	8,	-3,	-5,	-11,	-9,	-12,	-8,	0,	1]
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]

Q1. 13:00のデータのみを取り出すには？

input

a[2]

output

15

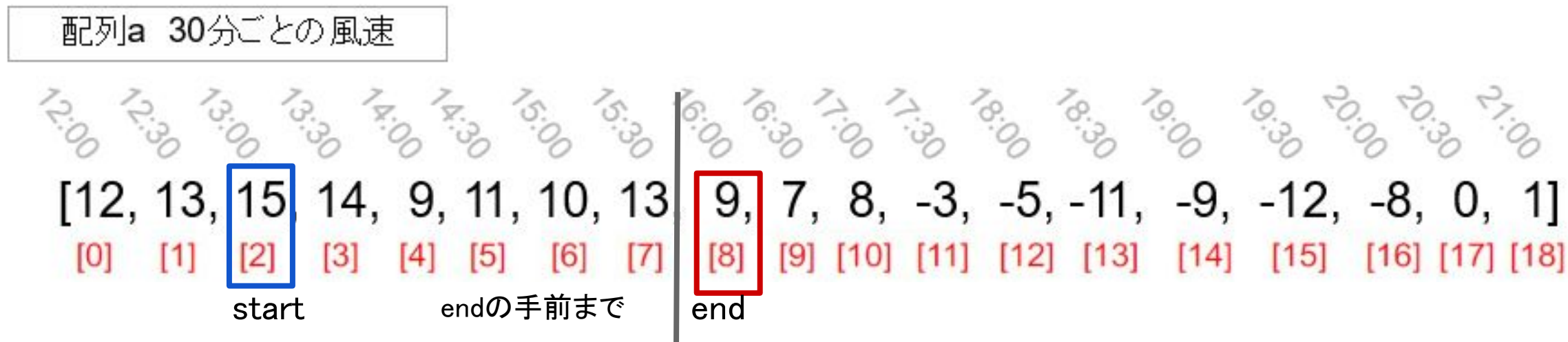
冒頭のクイズの答え

配列a 30分ごとの風速

12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00	20:30	21:00
12	13	15	14	9	11	10	13	9	7	8	-3	-5	-11	-9	-12	-8	0	1
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]

Q2. 13:00~16:00(未満)のデータのみを取り出すには？

冒頭のクイズの答え



Q2. 13:00~16:00(未満)のデータのみを取り出すには？

input

a[2: 8]

output

array([15, 14, 9, 11, 10, 13])

冒頭のクイズの答え

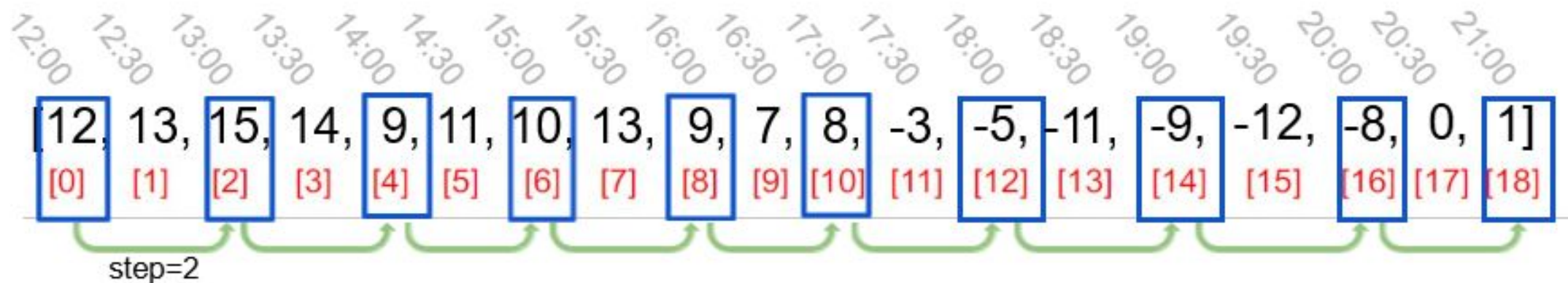
配列a 30分ごとの風速

12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00	20:30	21:00
[12,	13,	15,	14,	9,	11,	10,	13,	9,	7,	8,	-3,	-5,	-11,	-9,	-12,	-8,	0,	1]
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]

Q3. 00分ちょうどのデータのみを取り出すには？

冒頭のクイズの答え

配列a 30分ごとの風速



Q3. 00分ちょうどのデータのみを取り出すには？

input

`a[:: 2]`

output

`array([12, 15, 9, 10, 9, 8, -5, -9, -8, 1])`

ブール値(True・False)によって取得したい値を指定することもできます

a	1	1	2	3	5	8	13	21
---	---	---	---	---	---	---	----	----

$\text{idx} = (a \% 3 == 0)$

※a を3で割って余りが0になる＝割り切れるものを抽出

idx	False	False	False	True	False	False	False	True
-----	-------	-------	-------	------	-------	-------	-------	------

$\text{print}(a[\text{idx}])$

1	1	2	3	5	8	13	21
False	False	False	True	False	False	False	True

[3,21]

少しNotebookをみておきましょう

ブール値(True・False)によって取得したい値を指定することもできます

a	1	1	2	3	5	8	13	21
---	---	---	---	---	---	---	----	----

& : かつ
| : または

idx = (a % 3 == 0) & (a > 10)

※ 3で割り切れて、かつ10より大きいものを抽出

idx	False	False	False	False	False	False	False	True
-----	-------	-------	-------	-------	-------	-------	-------	------

print(a[idx])

1	1	2	3	5	8	13	21
False	False	False	False	False	False	False	True

[21]

いま説明したのが赤枠部分

(本日の講義でできるようになること)

Numpyと呼ばれるライブラリ(ツールのようなもの)の基本機能が使えるようになる

1. Numpyとは何か、その特徴

2. 1次元配列

2-1. 計算の基本(ユニバーサル関数・ブロードキャスト・集約関数)

2-2. 中身のデータ参照方法(インデクシング)

3. 2次元配列

3-1. 計算の基本(ユニバーサル関数・ブロードキャスト)

3-2. 縦軸・横軸の概念(axis・集約関数)

3-3. 中身のデータ参照方法(インデクシング)

3.二次元配列

numpy.ndarrayで、**N次元配列データ**を表す

2次元以上でもndarrayを作ることができます。

1次元の場合と同様に、基本的には以下のように作成します。

input

```
a = np.array([ [5, 6, 8], [9, 2, 1] ])
```

ネスト構造のリスト

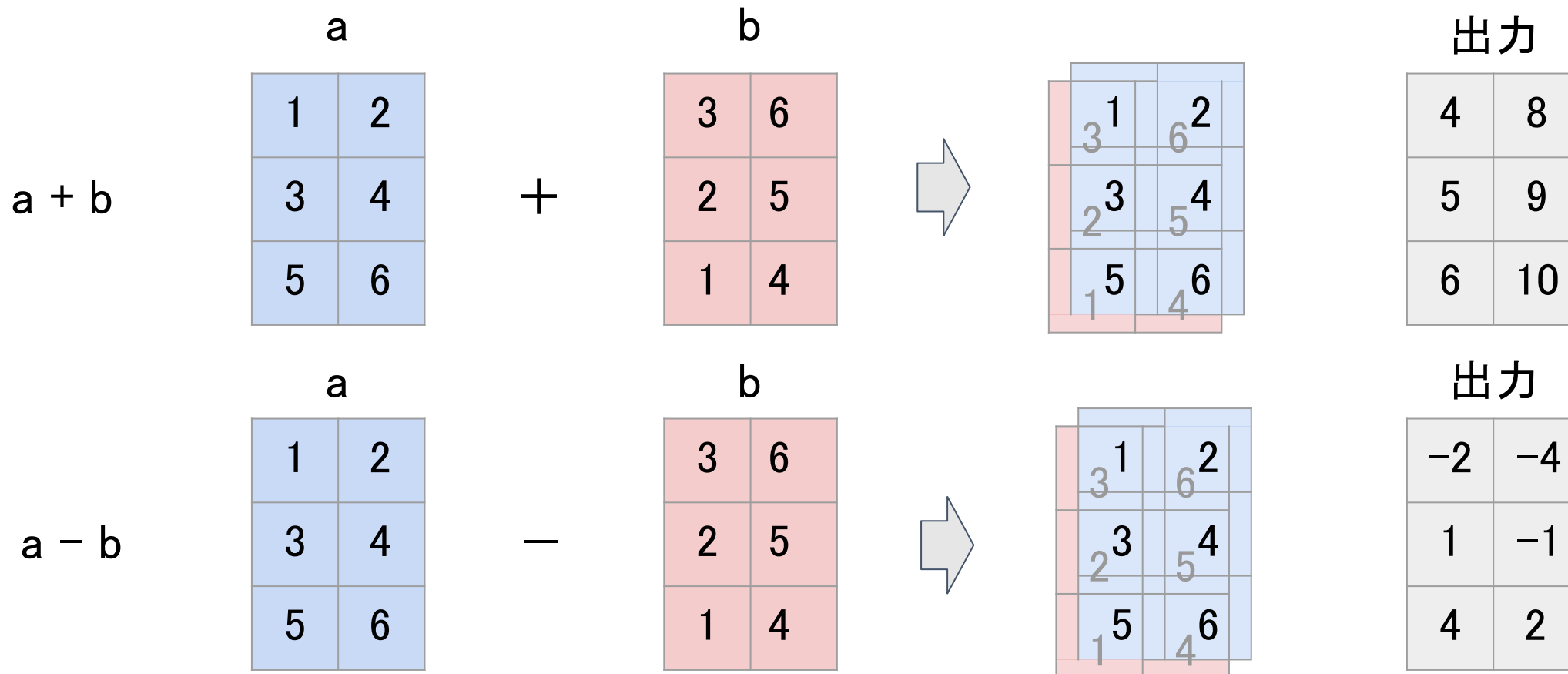
```
a =      np.array([[5,  6, 8],  
                  [9,  2, 1]])
```

という書き方でもOK
(改行は実行に影響ありません)

ネスト構造: リストの中にリストが入っている構造

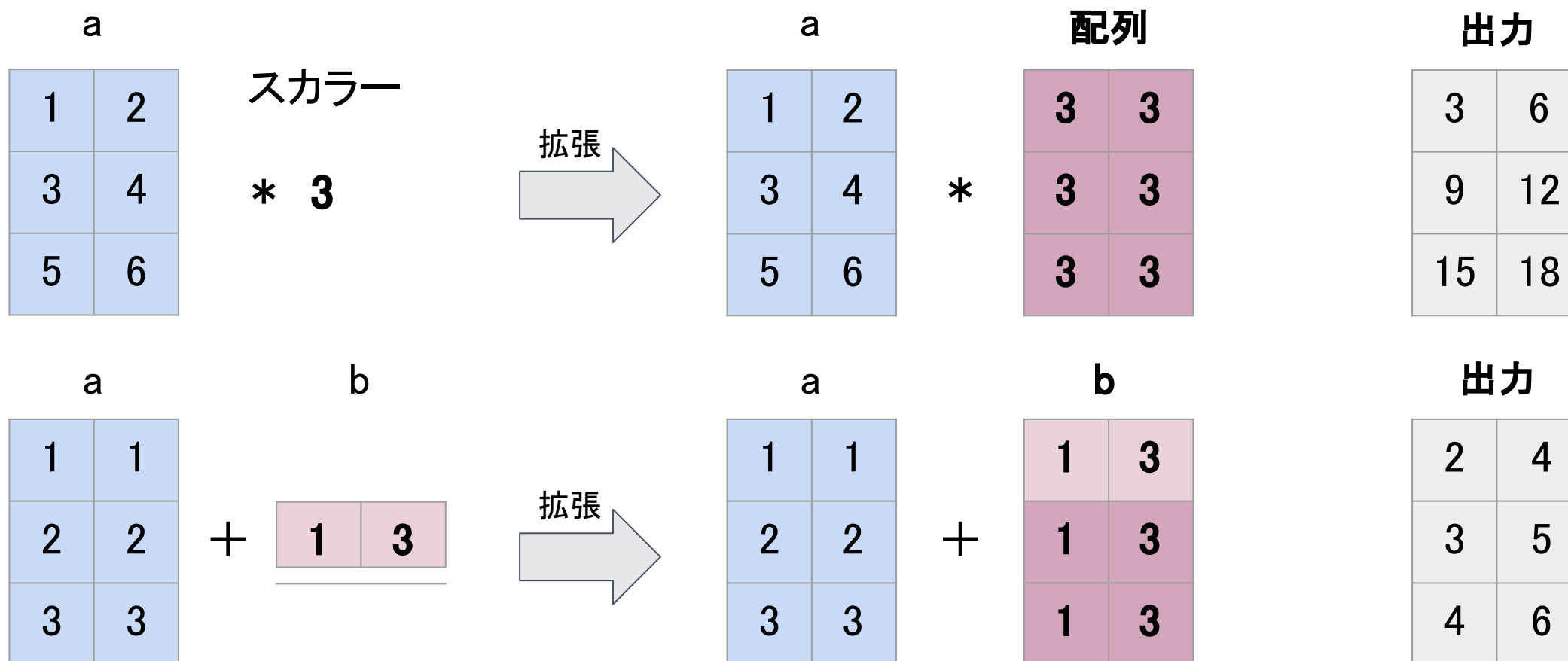
ユニバーサル関数(ufunc): ndarrayを要素ごとに操作する関数

2次元配列でも、要素ごとの四則演算を行うことができます



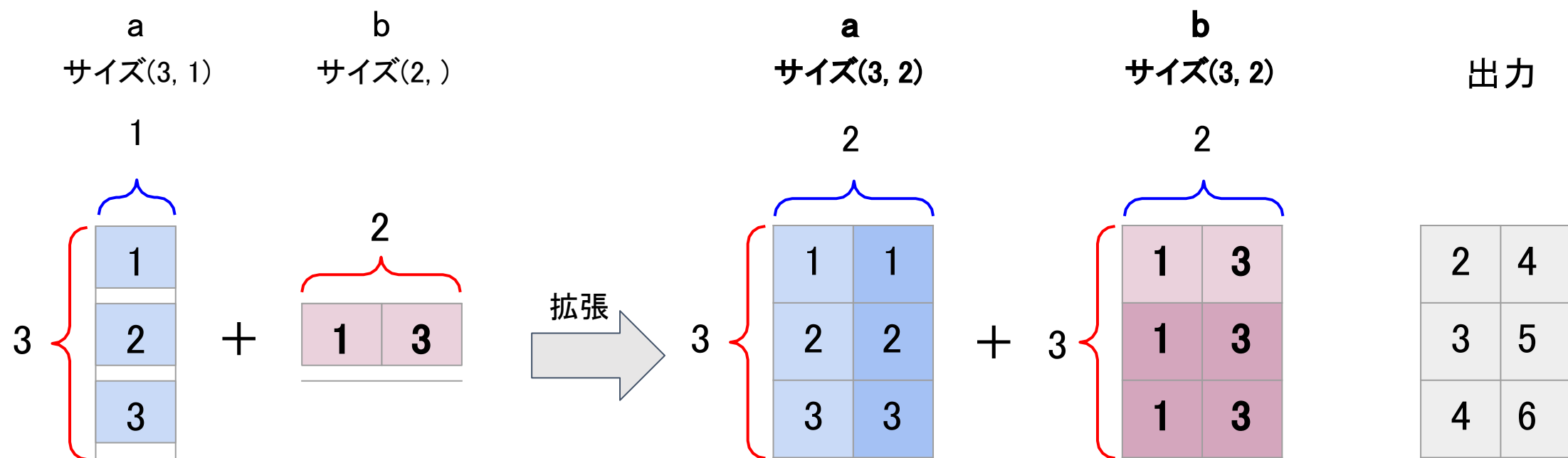
ブロードキャスト: 自動的に形状を合わせて演算を可能にする機能

サイズが1の次元が、相手の配列の同じ位置の次元のサイズに拡張されます。



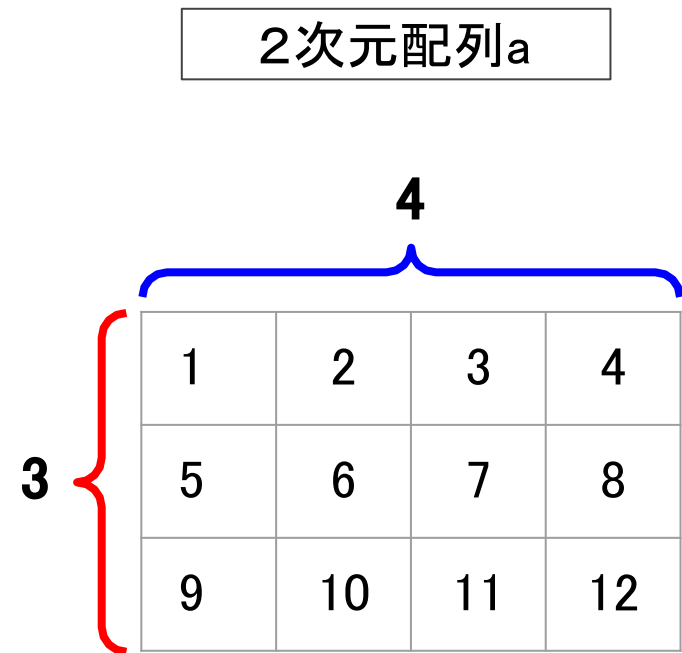
ブロードキャスト: 自動的に形状を合わせて演算を可能にする機能

サイズが1の次元が、相手の配列の同じ位置の次元のサイズに拡張されます。



※ 1次元配列には内部で次元が追加され、(2,) → (1, 2)の2次元配列に変換されてから拡張される

属性: ndarrayに関連付けられた情報



ndarray.shape: 配列の形状を示す

input

a.shape

output

(3, 4)

ndarray.ndim: 配列の次元数を示す

input

a.ndim

output

2

ndarray.dtype: 配列のデータタイプを示す

input

a.dtype

output

dtype('int64')

※ int64: 64bit整数型

縦軸・横軸の数値を指定して配列のサイズを調整できる

2次元配列a

4			
1	2	3	4
5	6	7	8
9	10	11	12



reshape

2次元配列b

6					
1	2	3	4	5	6
7	8	9	10	11	12

input

```
b = a.reshape(2, 6)
```

```
b.shape
```

output

```
(2, 6)
```

(2, -1)を指定すると(2, 6)に自動変換される

input

```
b = a.reshape(2, -1)
```

```
b.shape
```

output

```
(2, 6)
```

Notebookは今お伝えしたことの繰り返しになるので **割愛**します
 復習の際にご活用ください

いま説明したのが赤枠部分

(本日の講義でできるようになること)

Numpyと呼ばれるライブラリ(ツールのようなもの)の基本機能が使えるようになる

1. Numpyとは何か、その特徴

2. 1次元配列

2-1. 計算の基本(ユニバーサル関数・ブロードキャスト・集約関数)

2-2. 中身のデータ参照方法(インデクシング)

3. 2次元配列

3-1. 計算の基本(ユニバーサル関数・ブロードキャスト)

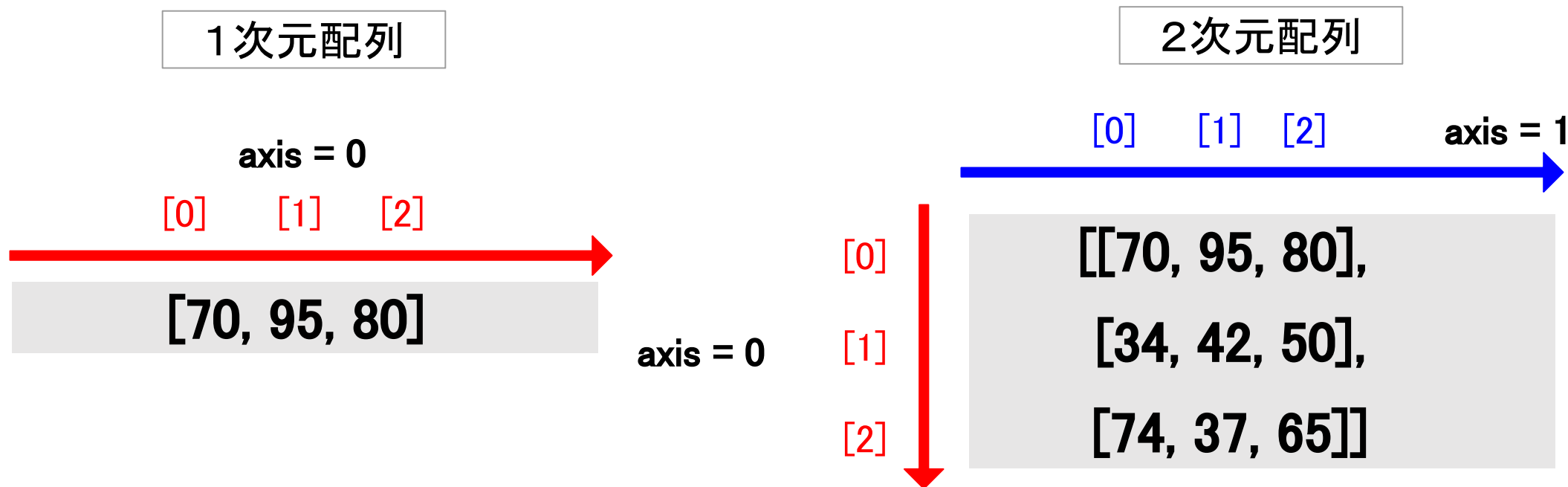
3-2. 縦軸・横軸の概念(axis・集約関数)

3-3. 中身のデータ参照方法(インデクシング)

axis: N次元配列に対して簡単に特定の方向にアクセスするための概念

ndarrayでは、axisという概念で軸方向が区別されます。

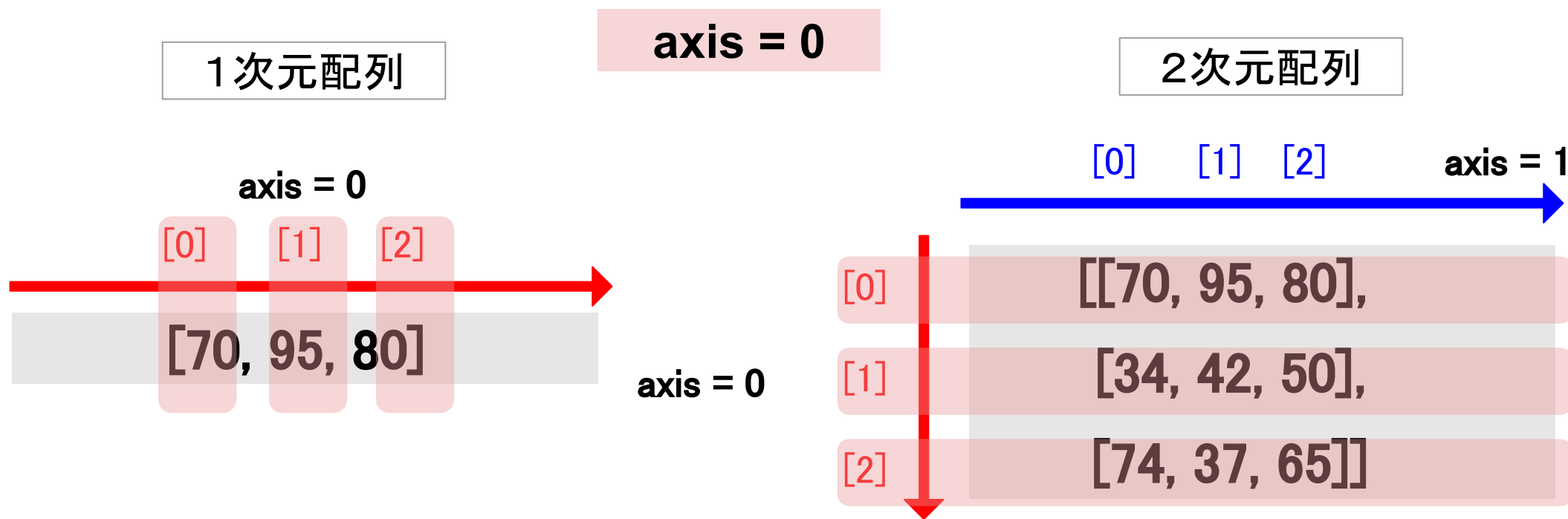
2次元配列の場合、axis=0は行、axis=1は列を表します。



axis: N次元配列に対して簡単に特定の方向にアクセスするための概念

ndarrayでは、axisという概念で軸方向が区別されます。

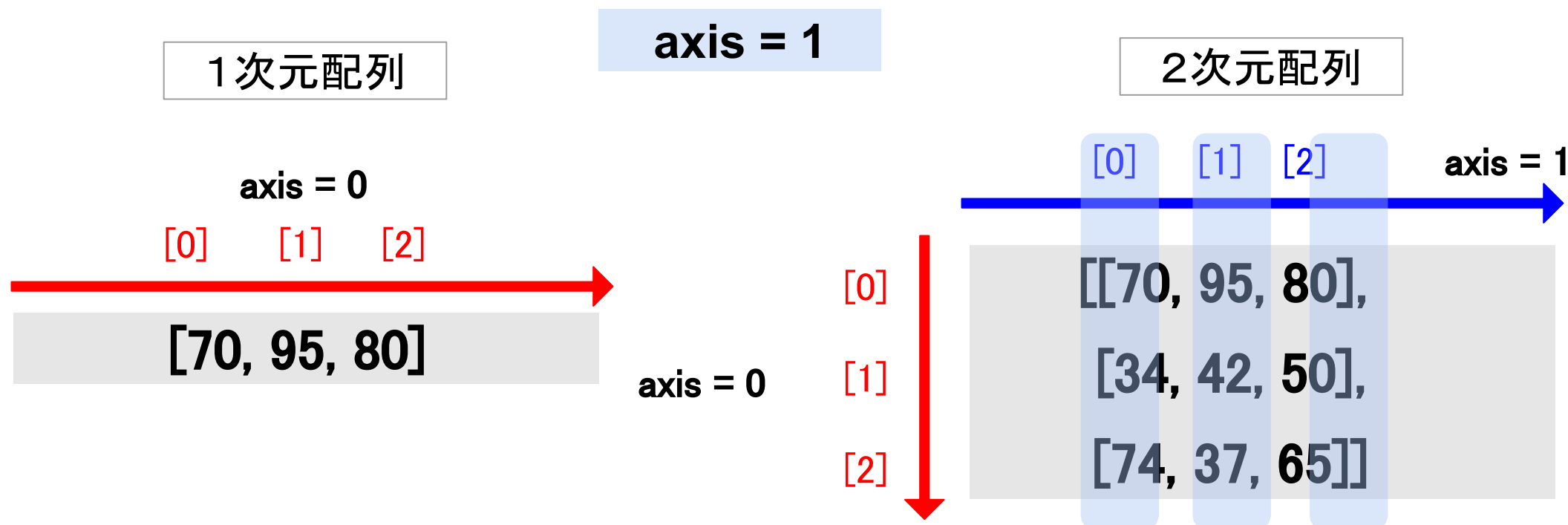
2次元配列の場合、axis=0は行、axis=1は列を表します。



axis: N次元配列に対して簡単に特定の方向にアクセスするための概念

ndarrayでは、axisという概念で軸方向が区別されます。

2次元配列の場合、axis=0は行、axis=1は列を表します。



集約関数: Numpyの配列内の要素を集約する関数

集約関数を用いて、各次元ごとの最大値を出してみましょう。

2次元配列 a

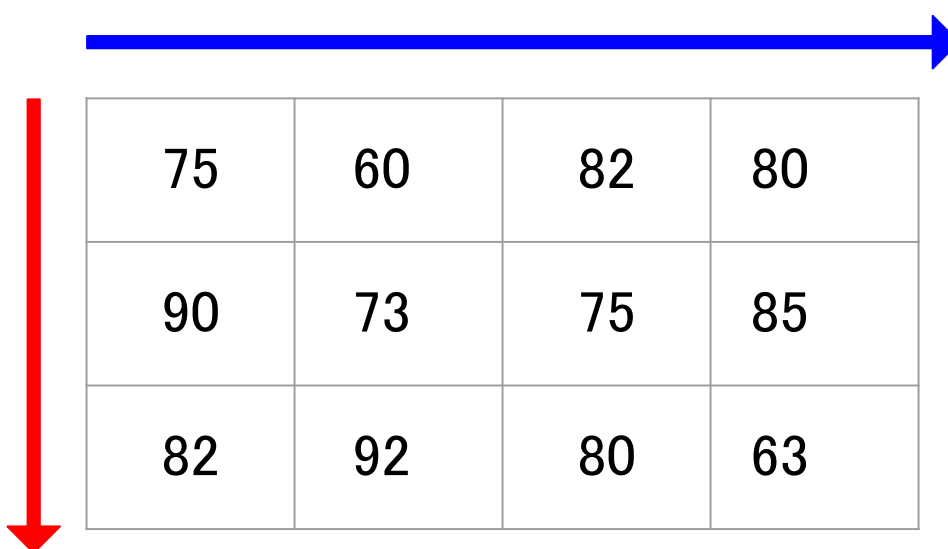
Aさん Bさん Cさん Dさん

[0] [1] [2] [3]

国語[0]

数学[1]

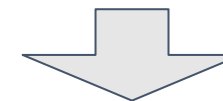
英語[2]



75	60	82	80
90	73	75	85
82	92	80	63

Q1. A～Dさんのそれぞれの
最高得点は？

Q2. 各教科の最高得点は？



集約関数で調べる

集約関数: Numpyの配列内の要素を集約する関数

2次元配列 a

axis = 1

[0] [1] [2] [3]

axis = 0

[0]

[1]

[2]

75	60	82	80
90	73	75	85
82	92	80	63

input

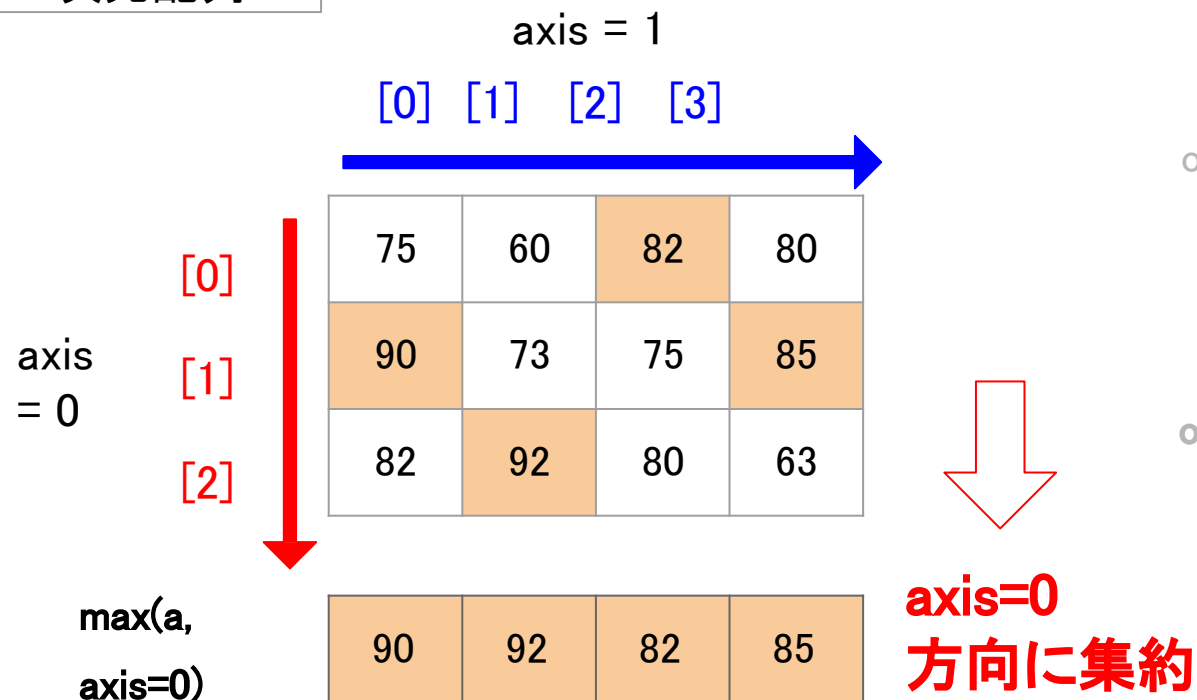
`np.max(a)`

output

92

`np.max(ndarray, axis=0)`のように軸指定して、axis=0方向の最大値を得る

2次元配列 a



input

`np.max(a)`

output

92

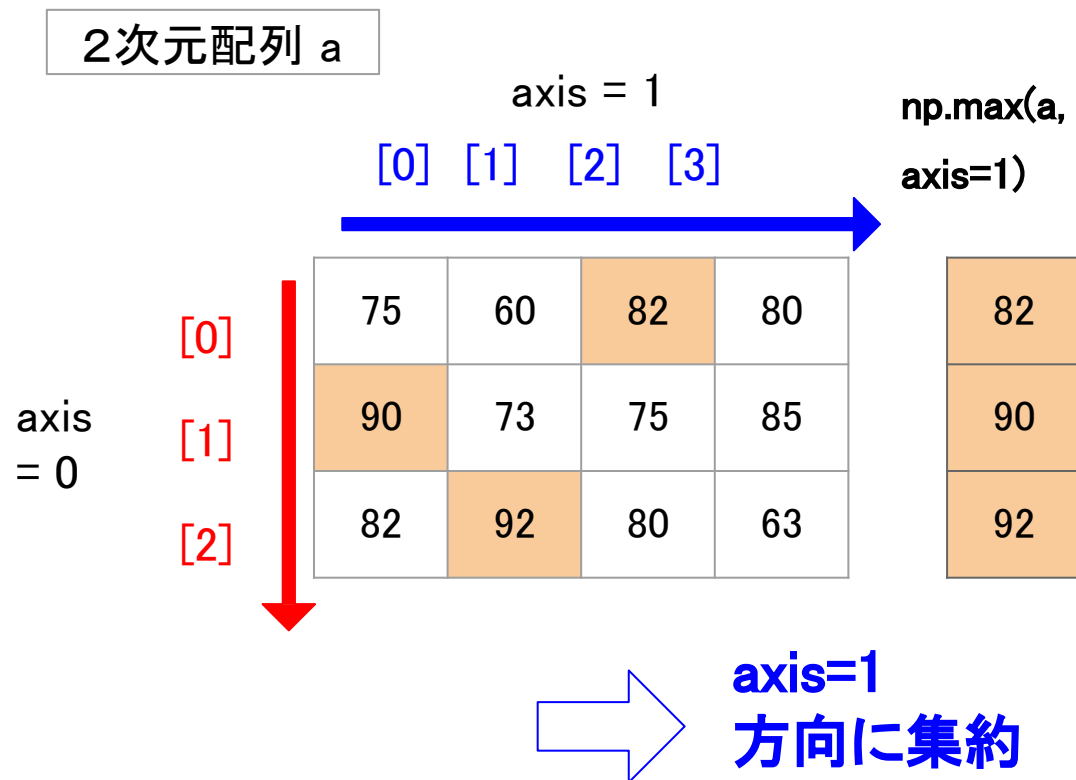
input

`np.max(a, axis=0)`

output

`array([90, 92, 82, 85])`

`np.max(ndarray, axis=1)`のように軸指定して、axis=1方向の最大値を得る



input `np.max(a)`

output 92

input `np.max(a, axis=0)`

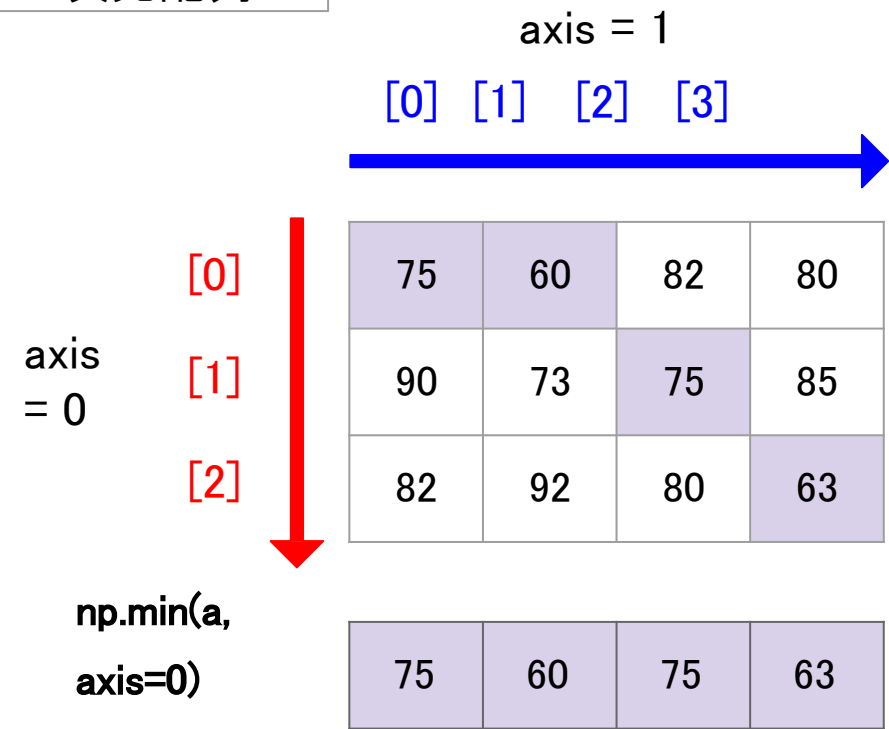
output `array([90, 92, 82, 85])`

input `np.max(a, axis=1)`

output `array([82, 90, 92])`

np.max()以外の集約関数 np.min()、np.sum()、np.mean()、np.std()

2次元配列 a



axis=0方向に集約

最小値

input

np.min(a, axis=0)

output

array([75, 60, 75, 63])

合計値

input

np.sum(a, axis=0)

平均値

input

np.mean(a, axis=0)

標準偏差

input

np.std(a, axis=0)

少しNotebookをみておきましょう

いま説明したのが赤枠部分

(本日の講義でできるようになること)

Numpyと呼ばれるライブラリ(ツールのようなもの)の基本機能が使えるようになる

1. Numpyとは何か、その特徴

2. 1次元配列

2-1. 計算の基本(ユニバーサル関数・ブロードキャスト・集約関数)

2-2. 中身のデータ参照方法(インデクシング)

3. 2次元配列

3-1. 計算の基本(ユニバーサル関数・ブロードキャスト)

3-2. 縦軸・横軸の概念(axis・集約関数)

3-3. 中身のデータ参照方法(インデクシング)

冒頭クイズインデクシング(2)

様々なインデックス指定方法で、2次元配列 a から任意の要素を抽出してみましょう。

2次元配列 a

	Aさん	Bさん	Cさん	Dさん	axis = 1
	[0]	[1]	[2]	[3]	
国語 [0]	75	60	82	80	
数学 [1]	90	73	75	85	
英語 [2]	82	92	80	63	

axis = 0

Q1. Dさんの数学の成績は？

Q2. A, B, Cさんの国語と英語の成績は？



インデクシングによって取り出す

2次元配列から、axis=0方向の要素を取り出す

Q1

2次元配列a

axis=1

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

axis=0

欲しい出力

[0]	[1]	[2]	[3]
1	2	3	4

input

a[0]

output

array([1, 2, 3, 4])

axis=1

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

axis=0

インデクシング(2) N次元配列の基本的なインデックス指定

低い次元からインデックスを指定して要素を取り出す(① axis=0 ② axis=1)

Q2

2次元配列a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

欲しい出力

[0]
2

①axis=0 ②axis=1

input

a[0, 1]

output

2

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

連続している要素はスライスで指定する

Q3

2次元配列a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

欲しい出力

	[0]	[1]	[2]
	2	6	10

↓ インデックスが連続しているので、スライスで指定できる

input

`a[:, 1]`

output

`array([2, 6, 10])`

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

連続している要素はスライスで指定する

Q4 ↓ インデックスが連続しているので、スライスで指定できる

2次元配列a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

欲しい出力

		[0]	[1]	[2]
axis=0	[0]	2	3	4
	[1]	6	7	8

input

a[:, 2, 1:]

output

array([[2, 3, 4], [6, 7, 8]])

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

このような複合的なインデクシングも可能

Q5

2次元配列a

axis=1

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

axis=0

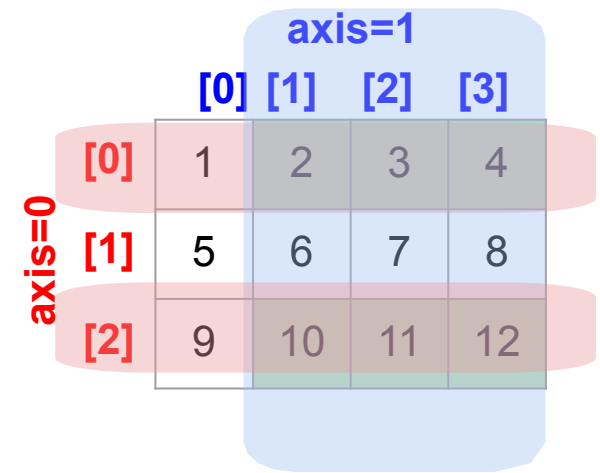
欲しい出力

	[0]	[1]	[2]
[0]	2	3	4
[1]	10	11	12

↓ 高度な指定 ↓ 基本的な指定

input `a[[0, 2], 1:]`

output `array([[2, 3, 4], [10, 11, 12]])`



冒頭クイズの答え

2次元配列 a

	Aさん	Bさん	Cさん	Dさん	axis = 1
	[0]	[1]	[2]	[3]	
					
国語 [0]		75	60	82	80
数学 [1]		90	73	75	85
英語 [2]		82	92	80	63
axis = 0					

Q1. Dさんの数学の成績は？

冒頭クイズの答え

2次元配列 a

	Aさん	Bさん	Cさん	Dさん	axis = 1
	[0]	[1]	[2]	[3]	
国語 [0]	75	60	82	80	input
数学 [1]	90	73	75	85	
英語 [2]	82	92	80	63	
axis = 0					output

Q1. Dさんの数学の成績は？

input a[1, 3]

output 85

冒頭クイズの答え

2次元配列 a

	Aさん	Bさん	Cさん	Dさん	axis = 1
	[0]	[1]	[2]	[3]	
国語 [0]	75	60	82	80	
数学 [1]	90	73	75	85	
英語 [2]	82	92	80	63	

axis = 0

Q2. A, B, Cさんの国語と英語の成績は？

冒頭クイズの答え

2次元配列 a

	Aさん [0]	Bさん [1]	Cさん [2]	Dさん [3]	axis = 1
国語 [0]	75	60	82	80	→
数学 [1]	90	73	75	85	
英語 [2]	82	92	80	63	
axis = 0	↓				

Q2. A, B, Cさんの国語と英語の成績は？

input

`a[[0, 2], : 3]`

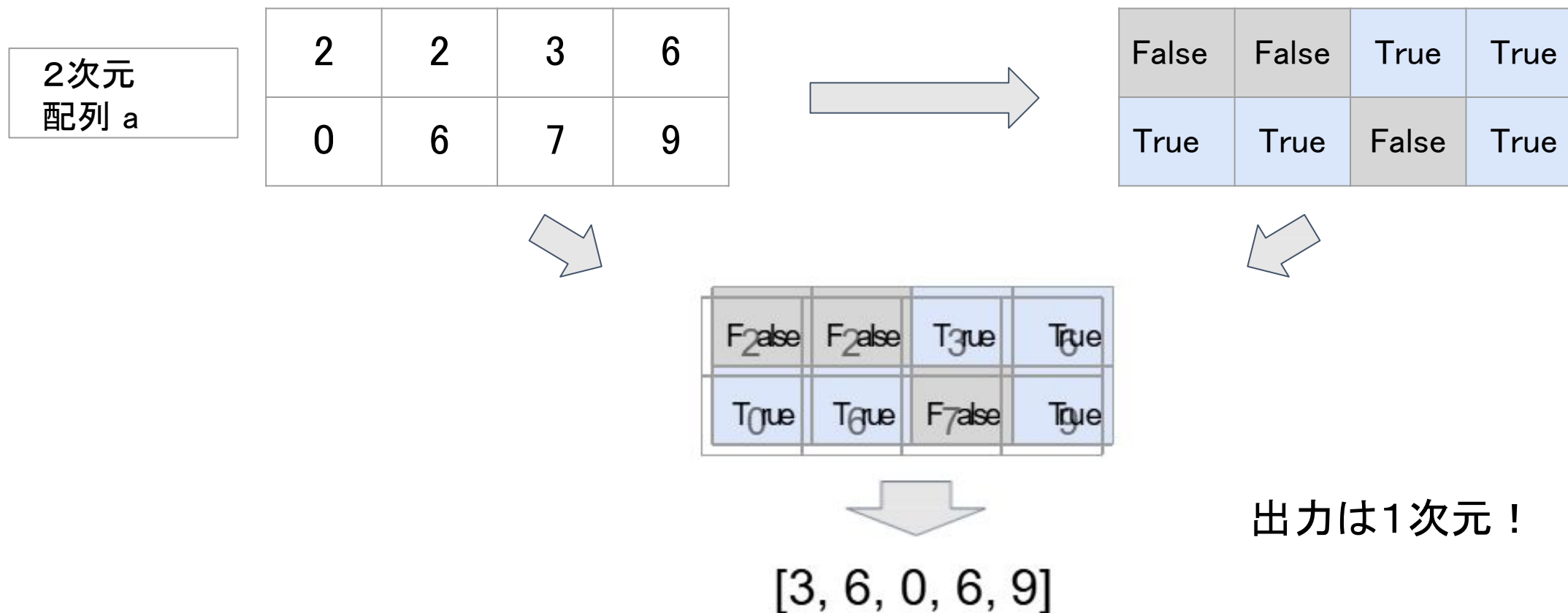
output

`array([[75, 60, 82],
[82, 92, 80]])`

ブール値によるインデックス指定

2次元配列でもブール値によって取得したい値を指定することができます。

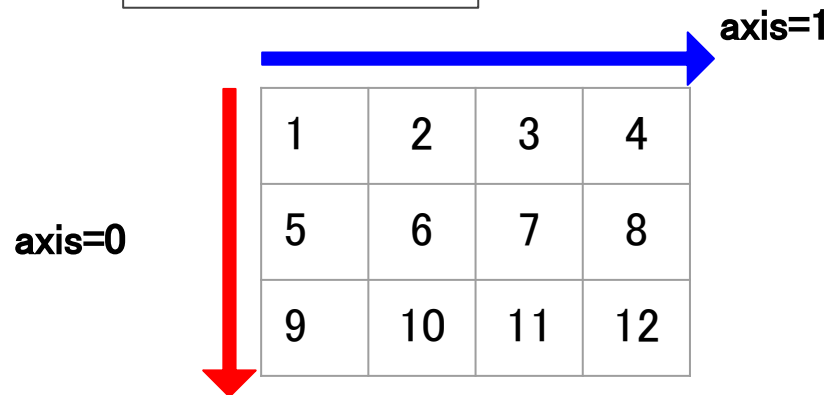
$$a \% 3 == 0$$



ndarray.T: 転置。配列の行と列を入れ替える操作

ndarray.T: 転置。配列の行と列を入れ替える操作

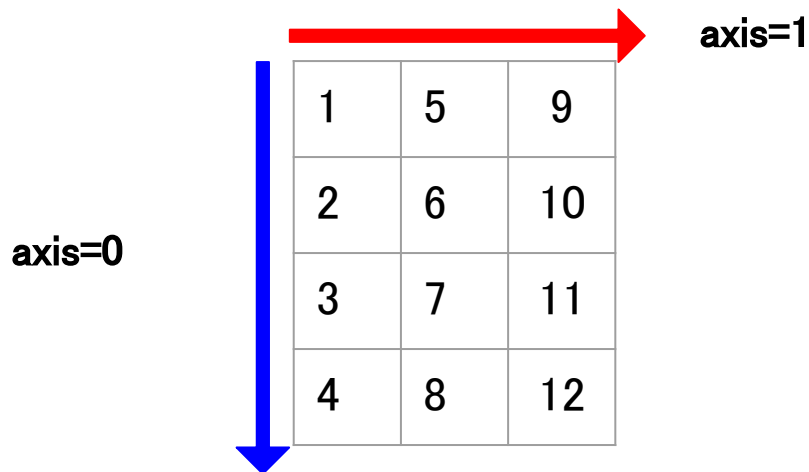
2次元配列a



A 4x4 grid representing array 'a'. A red arrow labeled 'axis=0' points downwards along the first column. A blue arrow labeled 'axis=1' points to the right along the first row.

1	2	3	4
5	6	7	8
9	10	11	12

2次元配列b (a.T)



A 3x4 grid representing array 'b'. A red arrow labeled 'axis=1' points to the right along the first row. A blue arrow labeled 'axis=0' points downwards along the first column.

1	5	9
2	6	10
3	7	11
4	8	12

input

```
b = a.T b
```

output

```
array([[ 1,  5,  9],  
       [ 2,  6, 10],  
       [ 3,  7, 11],  
       [ 4,  8, 12]])
```

input

```
b.shape
```

output

```
(4, 3)
```

Notebookは今お伝えしたことの繰り返しになる
かつ現状では理解必須ではない複雑な操作になるので **割愛**します
(ご興味ある方のみチャレンジしてみてください)

いま説明したのが赤枠部分

(本日の講義でできるようになること)

Numpyと呼ばれるライブラリ(ツールのようなもの)の基本機能が使えるようになる

1. Numpyとは何か、その特徴

2. 1次元配列

2-1. 計算の基本(ユニバーサル関数・ブロードキャスト・集約関数)

2-2. 中身のデータ参照方法(インデクシング)

3. 2次元配列

3-1. 計算の基本(ユニバーサル関数・ブロードキャスト)

3-2. 縦軸・横軸の概念(axis・集約関数)

3-3. 中身のデータ参照方法(インデクシング)

Numpy: Pythonで複雑な科学計算を行うためのライブラリ

1. Numpyの特徴を理解できる
2. ユニバーサル関数によって、for文なしで要素ごとの計算ができる
3. ブロードキャストによって、サイズの違う配列同士で演算できる
4. ndarrayのインデックスやaxis(軸)の概念が分かる
5. インデクシングによって、配列から任意の要素や配列を取り出せる
6. 集約関数によって、axisごとの統計量を算出できる

★都度調べればよい

まずNumpyの機能についてどこまで覚える必要があるのか？ということですが、これに関しては、覚えるべきことは少ないと思っています。

特に今回の講義だと、1次元、2次元配列の基本的な扱い方を覚えるだけでよく、他の機能については**その都度調べればよい**のです。

そうすることで、自然とよく使うものは覚える一方、滅多に使わないようなものをたくさん覚える必要もなくなります。実際プロのエンジニアでも自分が普段扱わない機能については知らないことはたくさんあり、その都度調べています。

Omnicampusより出席アンケートの提出をお願いいたします。

各回の宿題・アンケートの締切日は受講の手引きをご確認ください。

アーカイブ動画は講義翌日以降に

受講の手引きにて視聴用のリンクを公開します。

また、公開はslackで通知します。

